

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze

8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I R. Indhu (192524332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Indhu

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation ; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Sequential File Organization vs Heap File Organization

Feature	Sequential File	Heap File
Organization	Records stored in a particular order	Records stored in no particular order
Insertion	Slow because correct position may be required	Very fast
Deletion	Moderate/slow	Easy
Retrieval	Fast for sequential access	Slow for searching
Searching	Efficient if data is ordered	Requires linear search
Range Queries	Efficient	Less efficient
Best use	Reports and ordered processing	Frequent insertions

Example

Sequential:

101 → 102 → 103 → 104

Heap:

103 → 101 → 104 → 102

Conclusion

Heap files are better when insertions are frequent, while **sequential files** are better when ordered retrieval and range processing are required.

Q2. Clustered vs Non-Clustered Indexing

Feature	Clustered Index	Non-Clustered Index
Data order	Data is physically/logically organized according to index key	Data remains separately organized
Number	Usually one per table	Multiple can exist
Range search	Very efficient	Efficient but may require additional lookups
Storage	Generally lower overhead	Requires separate index storage
Example	Employee records sorted by EmployeeID	Index on EmployeeName

Example

Suppose:

EmployeeID | Name

101 | Arun

102 | Indhu

103 | Priya

If records are stored according to EmployeeID, it can be a **clustered index**.

A separate index:

Name → Record Address

Arun → 101

Indhu → 102

Priya → 103

is a **non-clustered index**.

Conclusion

Clustered indexing is useful for **ordered and range retrieval**, while non-clustered indexing is useful when **multiple search attributes** are required.

Q3. Primary Index vs Secondary Index

Feature	Primary Index	Secondary Index
Based on	Primary key	Non-primary attribute
Uniqueness	Usually unique	May contain duplicate values
Number	Generally one	Multiple possible
Storage	Lower	Higher if many indexes are created
Search	Very fast	Fast
Example	StudentID	Department

Example

Primary index:

StudentID → Record

101 → Student A

102 → Student B

103 → Student C

Secondary index:

Department → Records

CSE → 101, 103

ECE → 102

Conclusion

Primary indexing generally requires **less index space** and provides fast unique-key retrieval. Secondary indexing needs additional storage but allows searching using other attributes.

Q4. Static Hashing vs Dynamic Hashing

Feature	Static Hashing	Dynamic Hashing
Number of buckets	Fixed	Can change
Database growth	Less suitable	Highly suitable
Overflow	Overflow chains may occur	Handled dynamically
Scalability	Low	High
Implementation	Simple	More complex
Example	Fixed 10 buckets	Extendible/Linear hashing

Static Hashing

Key



Hash Function



Fixed Bucket

If the bucket becomes full, overflow occurs.

Dynamic Hashing

Key



Hash Function



Bucket



Overflow



Bucket expansion/splitting

Conclusion

Static hashing is simple and suitable for stable data. **Dynamic hashing** is better when the number of records changes frequently.

Q5. B-Tree vs B+ Tree

Feature	B-Tree	B+ Tree
Data storage	Internal and leaf nodes	Mainly leaf nodes
Leaf linking	No	Yes
Search	Fast	Very fast
Range queries	Good	Excellent
Sequential access	Less efficient	Very efficient
Database usage	Less common	Very common
Large databases	Good	Better

B-Tree

[20]

/ \

[10,15] [25,30]

Data can exist in internal nodes.

B+ Tree

[20]

/ \

[10,15] → [20,25,30]

Leaf nodes are linked.

Conclusion

For large-scale database applications, **B+ Tree is generally preferred** because it provides efficient search, sequential access, and range queries.

Q6. Linear Hashing vs Extendible Hashing

Feature	Linear Hashing	Extendible Hashing
Directory	No directory required	Uses directory
Bucket expansion	Gradual	Can involve directory doubling
Scalability	Good	Excellent

Bucket splitting	Progressive	Based on hash bits/depth
Memory overhead	Lower	Higher due to directory
Complexity	Moderate	Higher

Linear Hashing

Buckets are split **gradually**.

Bucket 1 → split

Bucket 2 → split

Bucket 3 → split

Extendible Hashing

Directory can double:

Before:

00

01

After:

00

01

10

11

Conclusion

Linear hashing provides smooth gradual growth, while **extendible hashing** provides fast scalability through directory and bucket splitting.

Q7. Dense Index vs Sparse Index

Feature	Dense Index	Sparse Index
Index entries	Every record	Selected records
Memory usage	High	Low
Access speed	Very fast	Slightly slower
Storage requirement	More	Less
Searching	Direct	May require additional scanning
Suitable for	Frequently searched	Large sorted files

Dense Index

Record → Index

101 → 101

102 → 102

103 → 103

104 → 104

Every record has an index entry.

Sparse Index

101 → Index

103 → Index

Only selected records have entries.

Conclusion

Dense indexing provides faster access but consumes more memory. **Sparse indexing** saves memory but may require additional searching.

Q8. Indexed Sequential File vs Direct File Organization

Indexed Sequential File Organization

Records are maintained sequentially and an index is used for faster access.

Index

↓

Sequential File

Advantages

- Supports sequential access.
- Supports direct access using index.
- Good for range queries.
- Efficient for ordered data.

Limitations

- Requires extra index storage.
- Updating the file can be more complex.
- Index maintenance is required.

Direct File Organization

A hash function directly determines the record location.

Key



Hash Function



Record Location

Advantages

- Very fast exact-match search.
- Fast insertion.
- Good for direct access.

Limitations

- Poor for range queries.
- Collision handling is required.
- Performance depends on hash function.

Conclusion

Indexed sequential organization is better for ordered and range access, while **direct organization** is better for fast exact-match access.

Q9. Single-Level vs Multi-Level Indexing

Feature	Single-Level Index	Multi-Level Index
Number of levels	One	Multiple
Search speed	Good	Better for very large files
Index size	Can become large	Each level reduces search area
Storage	More at one level	Organized across levels
Suitable for	Small/medium files	Large databases

Single-Level

Index



Data File

Multi-Level

Level 1 Index



Level 2 Index

↓
Level 3 Index

↓
Data File

Example

Instead of searching a huge index containing millions of entries, the first-level index points to a smaller second-level index.

Conclusion

Multi-level indexing is more efficient for large databases because it reduces the number of index entries that need to be examined during searching.

Q10. Hashing vs Indexing for Exact-Match and Range Queries

This question asks us to **evaluate**, so we need to decide which technique performs better for different query types.

Query Type	Hashing	Indexing
Exact-match	Excellent	Very good
Range query	Poor	Excellent
Search by key	Very fast	Fast
Ordered retrieval	Poor	Excellent
Storage	Moderate	Depends on index
Best example	Find AccountID = 1001	Find transactions from Jan–March

Exact-Match Query

Example:

```
SELECT *
```

```
FROM Student
```

```
WHERE StudentID = 101;
```

Hashing is very effective because:

```
StudentID
```

↓
Hash Function



Bucket



Record

Average search can be close to **O(1)** with a good hash table and controlled load factor.

Range Query

Example:

SELECT *

FROM Student

WHERE CGPA BETWEEN 8.0 AND 9.0;

A **B+ Tree index** is better because values are ordered:

8.0 → 8.1 → 8.2 → 8.3 → ... → 9.0

The linked leaves allow efficient sequential retrieval of the matching range.

Final Evaluation

Exact Match → Hashing is generally better

Range Query → B+ Tree Indexing is generally better

Therefore, a database can use **both techniques together** depending on the workload.